Project Requirements

Name: Robert Nolan

Student Number: G00474404

Module: Web Application Development

Table of Contents

Name: Robert Nolan..... 1
Student Number: G00474404..... 1
Module: Web Application Development..... 1
Item 1 – Customer Login..... 3
User Credentials..... 3
Allow the customer to enter their login details:..... 3
Login details validated before receiving order summary:..... 4
How this was achieved:..... 4
Item 2 – Form Validation..... 7
Checkout Page Form:..... 7
How this was achieved:..... 7
Item 3 – Slideshow / Carousel..... 10
Home Page Image Carousel:..... 10
How this was achieved:..... 10
Item 4 – Purchase Items..... 14
Add items to cart:..... 14
Stored in Local Memory:..... 15
Place Order and Receipt:..... 16
How this was achieved:..... 17
Item 5 – Object / Array in Javascript..... 21
Array use in Carousel:..... 21
How this was achieved:..... 21
Item 6 – Custom Node Module..... 23
Two Custom Node Modules:..... 23
How this was achieved:..... 23
Item 7 – Post and Get Requests..... 26
Get Requests:..... 26
Post Requests:..... 26
How this was achieved:..... 27
Item 8 – Static and Dynamic Content..... 30
Static Content:..... 30
Dynamic Content:..... 31
How this was achieved:..... 31
Item 9 – Templates in Node..... 36
Node Templates:..... 36
How this was achieved:..... 36
Item 10 – Error Message..... 38
Error handling:..... 38
How this was achieved:..... 39
Item 11 – MySQL Database..... 42
MySql:..... 42
How this was achieved:..... 42
Item 12 – Bootstrap Version 5..... 44
Bootstrap:..... 44
How this was achieved:..... 44
Additional..... 45
Folder Structure Layout..... 45
Image References..... 46

Item 1 – Customer Login

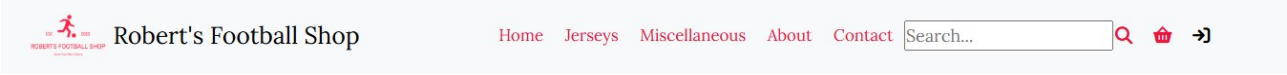
User Credentials

Username: user@123.com

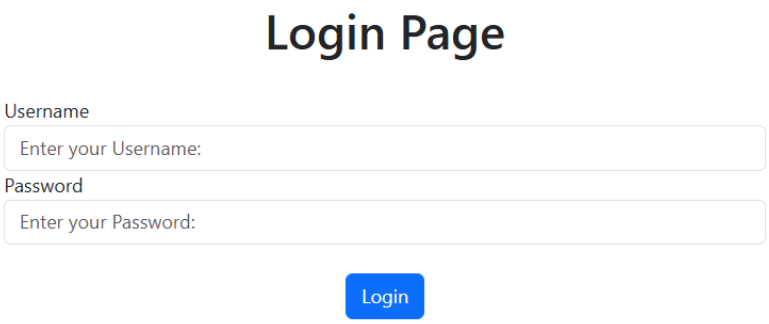
Password: pass (all lowercase)

Allow the customer to enter their login details:

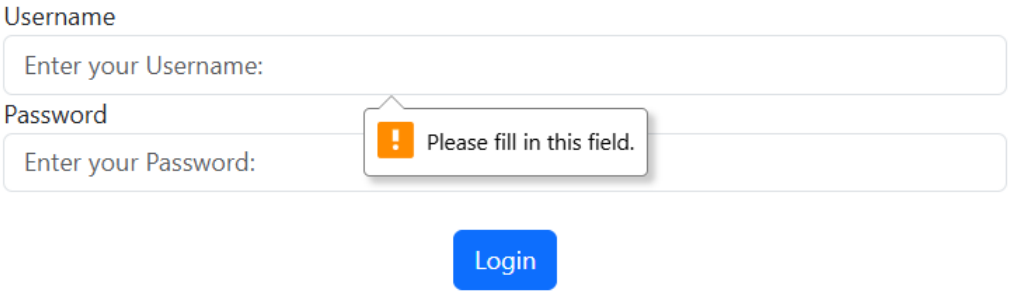
In the navbar on each page there is a login icon (shown in black in image below) which directs the user to the login page.



The login page is laid out so the user can input their username and password.



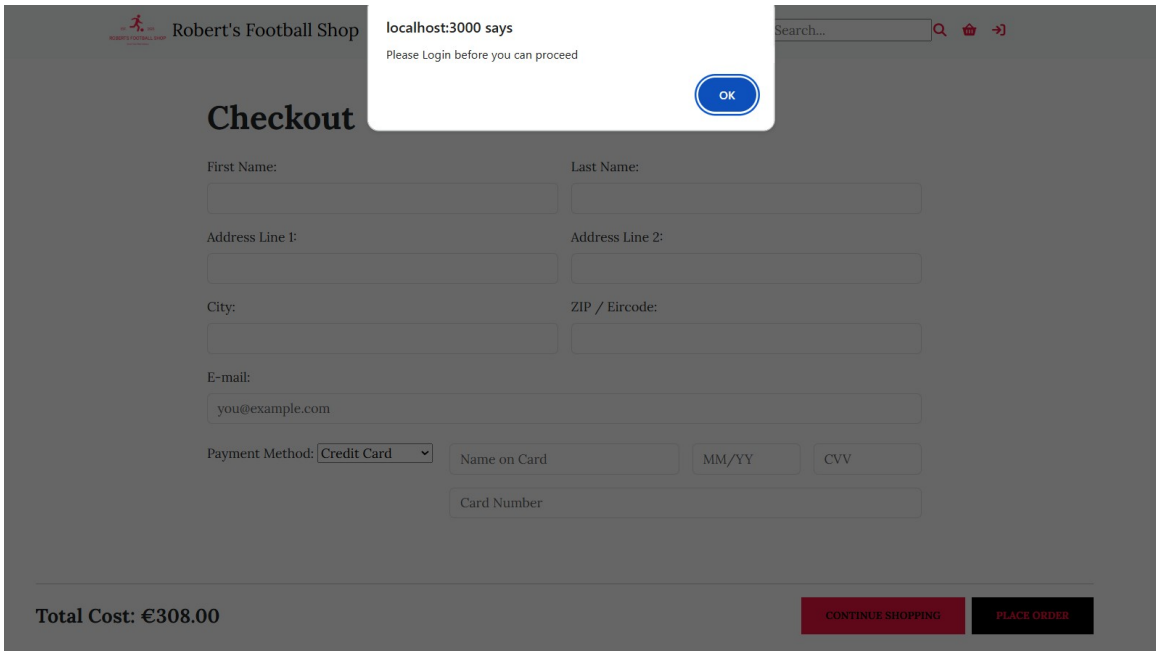
Credentials must be filled in before clicking the login button.



Once logged in the icon in the navbar changes to show the user is logged in. (red person with check)

Login details validated before receiving order summary:

On the checkout page if the user clicks Place Order but they are not logged in then a popup message will appear to alert them they must Login before they can proceed.



How this was achieved:

The auth.js custom node module is used to ensure authentication.

An in-memory array is used to store user data – const users = [ ]

The createUser function allows for user creation.

User verification is done through the authenticateUser function.

custom_node_modules > JS auth.js > ...

```
//create array for storing user data
const users = [];

function createUser(username, password) {
    users.push({username, password});
    console.log(users);
}

function authenticateUser(username, password) {
    //find user by username in array
    const checkUser = users.find(user => user.username === username);

    //if statement to check if user is within array and password is correct
    if (!checkUser || checkUser.password !== password) {
        return false
    }
    return true;
}

module.exports = {createUser, authenticateUser};
```

Username and password are saved in the app.js file.

```
//-------------------------------
//Create Users for the application
auth.createUser("user@123.ie", "pass")

//test users work
console.log(auth.authenticateUser("user@123.ie", "pass"));

// End of application users
//------------------------
```

When the user clicks the Login button on login.ejs the form sends the username and password to the /login route.

The POST request for authenticate in app.js makes use of auth,js to validate if username and password are correct.

If they are then it logs in, otherwise an error is displayed.

The user is stored in req.session.user so the website can show logged in on all pages after the user goes through the login process.

```
JS app.js > ...
//Route to handle Login Form Submission - (POST METHOD)
app.post("/login", function (req, res) {
    const username = req.body.username;
    const password = req.body.password;

    //authenticate the username and password
    const authenticated = auth.authenticateUser(username, password);
    console.log(authenticated);

    if (authenticated) {
        // store user in session
        req.session.user = username;

        //Logs to the console that the login was a success
        //If login is successful then directed to the home page (/)
        console.log("Successful Authentication");
        res.redirect("/");
    }
    else {
        //If login fails the login page renders again
        //Logs to the console that the login failed
        //Redirects to the login page with the previously input username still shown
        //Gives an alert message saying invalid username or passowrd
        console.log("Failed Authentication");
        res.render("login", {error: "Invalid username or password", username: username });
    }
});
```

In the checkout page a function checks if the user is logged in. This is part of the ‘Place Order’ button, meaning the user must be logged in before they can place their final order. It works with a click event listener in JavaScript and will give an alert if the user session is not shown as logged in.

```javascript
// ------------------------------
// Login check before order submit
// ------------------------------
const placeOrderBtn = document.getElementById("placeOrderBtn");

// This variable comes from ejs res.locals.user (sessions)
const loggedInUser = "<%= user ? user : '' %>";

// If user is not logged in then the alert pops up and then cannot proceed via preventDefault
placeOrderBtn.addEventListener("click", (e) => {
    if (!loggedInUser) {
        e.preventDefault(); // Stop form submission
        alert("Please Login before you can proceed");
    }

});

// -------------------------------------
// End of Login check before order submit
// -------------------------------------
```

In app.js express.session was imported which basically allows the website to know over multiple pages whether the user is logged in or not.

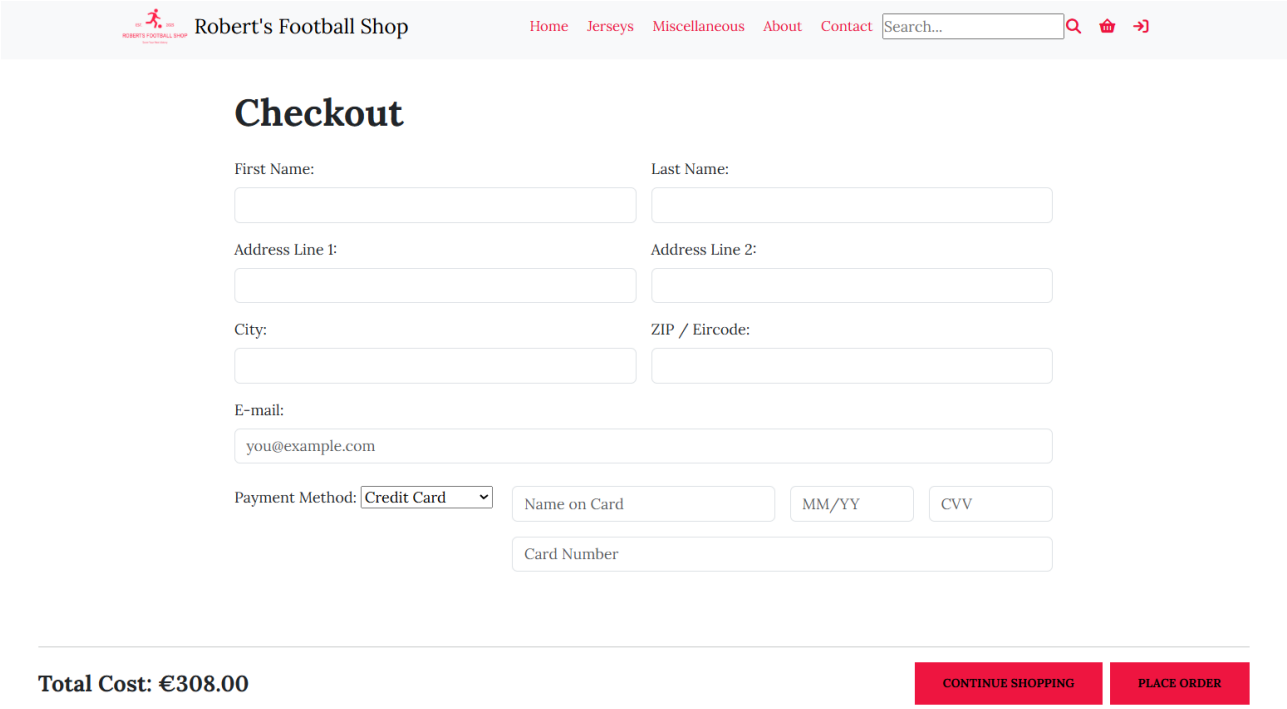
So if the user logs in on the home page, they will still be logged in when they navigate to the checkout page.

```javascript
//Uses express-session (for custom modules such as authentication of user and payment)
const session = require("express-session");

app.use(session({
    secret: "your-secret-key",
    resave: false,
    saveUninitialized: true
}));

// Makes user available to all pages if logged in
app.use((req, res, next) => {
    res.locals.user = req.session.user || null;
    next();
});
```

Item 2 – Form Validation

Checkout Page Form:

On the checkout page I have included the form validation. Users are required to input their details into the form, includng an email address in the email format, before they can submit the form via the Place Order button.



How this was achieved:

Within the checkout.ejs I have created the <form> tag which uses the action = /paymentSuccess and method POST. The action will redirect to the paymentSuccess.ejs page when the form is filled in, and the POST method is because this is considered sensitive data.

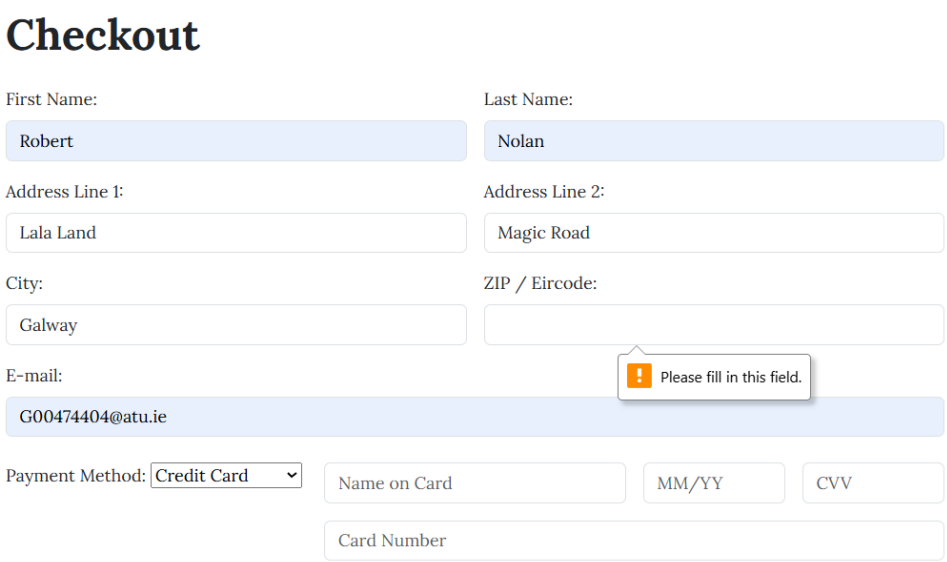
```
views > checkout.ejs > html > body > ? > div.container.mt-5 > div.container.mt-5 > form
 2   <html>
21   <body>
24       <%- include('templates/navbar') %>
35           <div class="container mt-5">
102              <div class="container mt-5" style="max-width: 900px;">
106                  <!--A Post method for submitting the form, redirects to paymentSuccess page-->
107                  <form action="/paymentSuccess" method="POST">
108
109
110                      <!------------------------------------------->
111                      <!-- TOP ROW: Shipping Address (full width) -->
112                      <!--
```

To ensure the form is filled in I added the required setting to the inputs.
These are on all the major input fields, and excluded from the non important ones such as Address Line 2.

For email I changed the type from a text requirement to an email requirement, meaning a valid email input must be include (ending in @xxxxx.xx).

```
views > checkout.ejs > html > body > ? > div.container.mt-5 > div.container.mt-5 > form > div.row.g-3.mb-4 > div.col-md-6
<html>
<body>
    <%- include('templates/navbar') %>
        <div class="container mt-5">
            <div class="container mt-5" style="max-width: 900px;">
                <div class="col-md-6">
                    <!-- name="addressLine2" = key for node.js server known as req.body.addressLine2 -->
                    <label class="form-label">Address Line 2: </label>
                    <input type="text" name="addressLine2" class="form-control">
                </div>

                <div class="col-md-6">
                    <!-- name="city" = key for node.js server known as req.body.city -->
                    <label class="form-label">City: </label>
                    <input type="text" name="city" class="form-control" required>
                </div>

                <div class="col-md-6">
                    <!-- name="zip" = key for node.js server known as req.body.zip -->
                    <label class="form-label">ZIP / Eircode: </label>
                    <input type="text" name="zip" class="form-control" required>
                </div>

                <!-- col-md-12 = column will use full width on all sizes -->
                <div class="col-md-12">
                    <!--
                        - name="email" = key for node.js server known as req.body.email
                        - type=email = ensures an email address is entered
                    -->
                    <label class="form-label">E-mail: </label>
                    <input type="email" name="email" class="form-control" required
                        placeholder="you@example.com">
                </div>
            </div>

            <!-- End of TOP ROW: Shipping Address -->
            <!------------------------------------------------------------>
```

If any of the required fields are left empty then this message appears:



Within the paymentSuccess.ejs the information input into the form as part of the POST request is used for the final receipt.

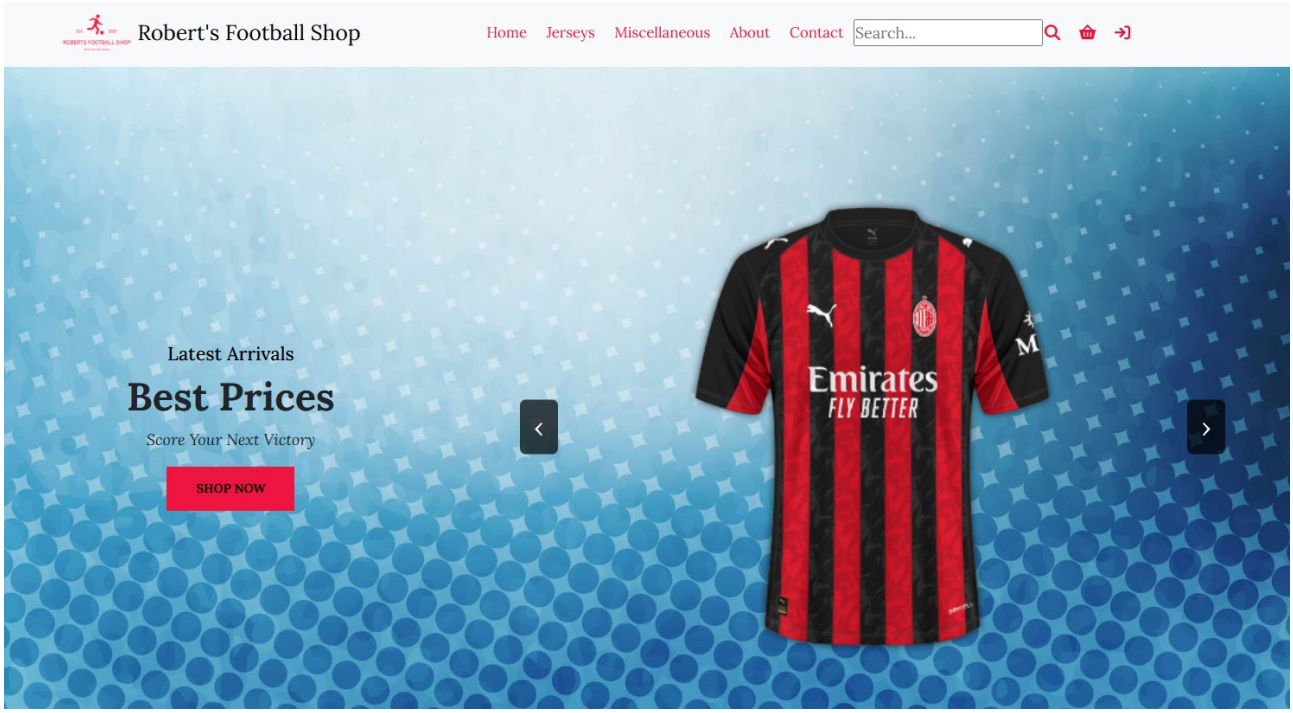
```
paymentSuccess.ejs > html > body > div#receipt > h3
<html>
<body>
    <div id="receipt">
        <!-- Customer Info -->
        <!-- Via POST Method on checkout.ejs form-->
        <h3>Customer Details</h3>
        <p>
            Name: <strong><%= formData.firstName %> <%= formData.lastName %></strong><br>
            Address: <strong><%= formData.addressLine1 %>
                <% if (formData.addressLine2) { %> <%= formData.addressLine2 %> <% } %>,
                <%= formData.city %>, <%= formData.zip %></strong><br>
            Email: <strong><%= formData.email %></strong><br>
            Payment Method: <strong><%= formData.paymentMethod %></strong>
        </p>
        <!-- End of Customer Info -->
```

Item 3 – Slideshow / Carousel

Home Page Image Carousel:

On the home page there is a carousel displaying products from the website (jerseys).
Every time the webpage is reloaded a new carousel of random images is created, with 10 images in the carousel each time.

It is applied to the right of the screen with image slider buttons either side, and on the left of the screen is a button redirecting to the shop page.



How this was achieved:

In app.js the app.get for the home page includes the code for the carousel.

The images are dynamically loaded to the server and randomised.

From there the images are stored in the images array for the carousel.

Within the random array of images the first ten are chosen for the carousel.

These are then sent to the home.ejs page to be displayed.

Everytime the home page is refreshed a new selection of images are stored in the carousel.

This way allows for automatic changes to the carousel images without needing to hardcode any image paths.

```js
app.js > ...
//-----------------------------
// For Home Page Carousel Images
app.get("/", (req, res) => {

    // Route to the saved images
    const imagesDir = path.join(__dirname, "public/images/jerseys");

    //Creates empty array for the images to be stored before shuffle
    let images = [];

    try {
        // Read all files in the folder
        images = fs.readdirSync(imagesDir)
            .filter(file => /\.(png|jpg|jpeg|webp)$/i.test(file)); // Only image files

        // Shuffle the array randomly
        for (let i = images.length - 1; i > 0; i--) {
            const j = Math.floor(Math.random() * (i + 1));
            [images[i], images[j]] = [images[j], images[i]];
        }

        // Picks the first 10 images after the shuffle
        images = images.slice(0, 10);

    } catch (err) {
        console.error("Error reading images folder:", err);
    }

    //Prints to the console the selected images
    console.log("Images selected for carousel:", images);

    //renders the home page with the carousel images
    res.render("home", { images });
});

//End of carousel
//---------------
```

Within home.ejs the images array is looped over to generate the carousel slides.

Each image is stored as a within the carousel.

Active is applied to the first image (index [0]), making it display.

```
-->
<div class="col-md-7">
    <div class="car">
        <div class="img-wrap">
            <% images.forEach((img, index)=> { %>
                <img src="/public/images/jerseys/<%= img %>"
                    class="carousel-img <%= index === 0 ? 'active' : '' %>" alt="Product image">
                <% }) %>
        </div>
        <!--Buttons to click next or previous image from the Carousel-->
        <!--
            - class btn prev = show previous image, used via Javascript
            - class btn next = show next image, used via Javascript
        -->
        <button class="btn prev">‹</button>
        <button class="btn next">›</button>
    </div>
</div>
<!-- End of Right side - Image Carousel-->
```

JavaScript then handles the automation and button controls from the carousel. It:

- selects all the images and buttons
- hides all images
- tracks what is currently visible

```
// Select all images used in the carousel
const images = document.querySelectorAll(".carousel-img");

// Select the buttons for previous and next for the carousel
const prevBtn = document.querySelector(".prev");
const nextBtn = document.querySelector(".next");

// Tracks images used in carousel array, starting at index 0
let currentIndex = 0;


// Function to show images at specific index
function showImage(index) {

    // Hides all images by removing the active class
    images.forEach(img => img.classList.remove("active"));

    // Only adds active to the selected images
    images[index].classList.add("active");
}
```

The previous and next buttons move through the index numbers of the carousel-img.

The next button adds to the current index, moving onto the next slide.

The previous button detracts from the current index, moving to the previous slide.

Event listeners are used for clicking on the buttons.

```
// For the clicking of the next button
nextBtn.addEventListener("click", () => {

    // Moves to next image and modulo makes index wrap back to 0 at end of array
    currentIndex = (currentIndex + 1) % images.length;

    // Displays the next image
    showImage(currentIndex);
});

// For the previous of the next button
prevBtn.addEventListener("click", () => {

    // Moves to previous image and modulo makes index wrap back to 0 at end of array
    // Adding images.length stops negative numbers
    currentIndex = (currentIndex - 1 + images.length) % images.length;

    // Displays the previous image
    showImage(currentIndex);
});
```

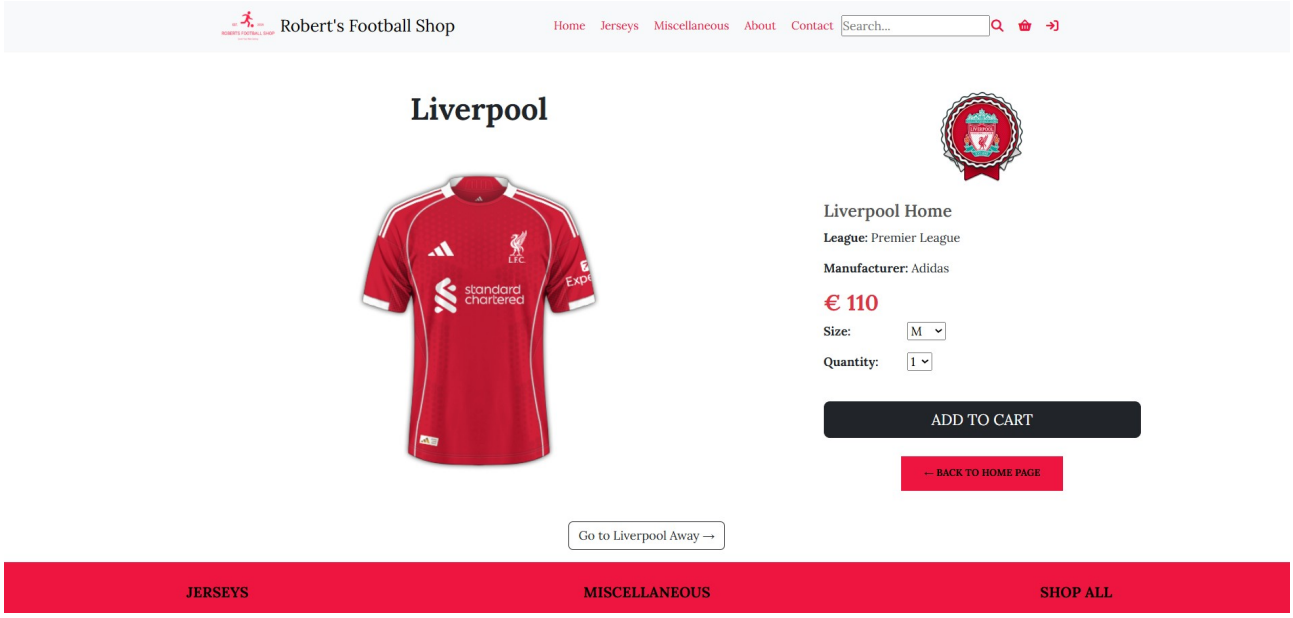
Finally an auto-play function is added to move through the index of images at an interval of 3 seconds.

```
    // Auto-play
    // Every 3 seconds the carousle automatically moves to the next image in the array
    setInterval(() => {
        currentIndex = (currentIndex + 1) % images.length;
        showImage(currentIndex);
    }, 3000);
</script>

</body>
```

Item 4 – Purchase Items

Add items to cart:

On every product page there is an ‘Add To Cart’ button. Users can choose their size and quantity before adding the desired product to the cart.

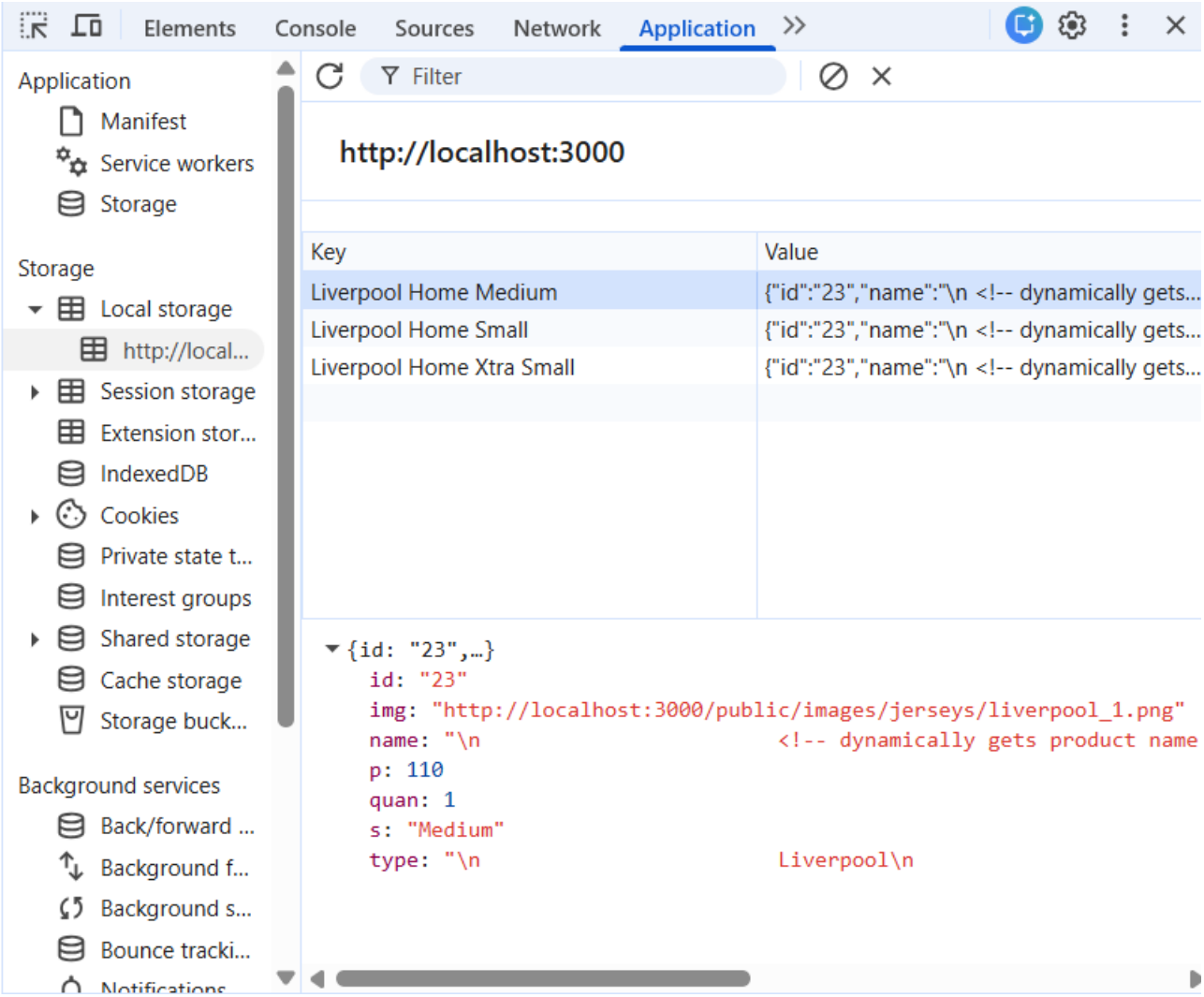


Stored in Local Memory:

When an item is added to the cart it is saved as a key (using JSON) to store the details (product ID, image, size, price, quantity, etc) in local memory.

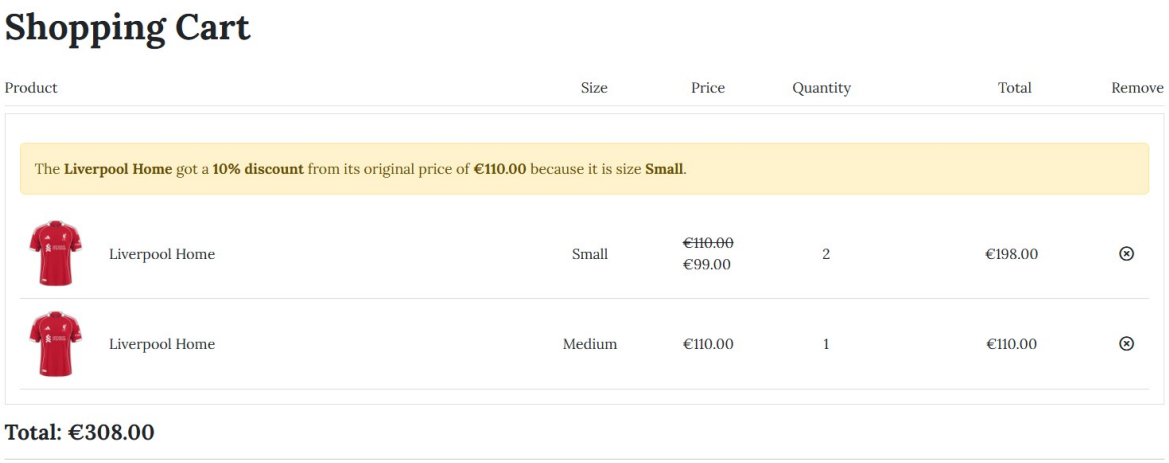
The information is stored in a way that allows users to add multiple of the same product with different sizes, with this information being used as the key.

The value then holds the required information for later use in the checkout.



Place Order and Receipt:

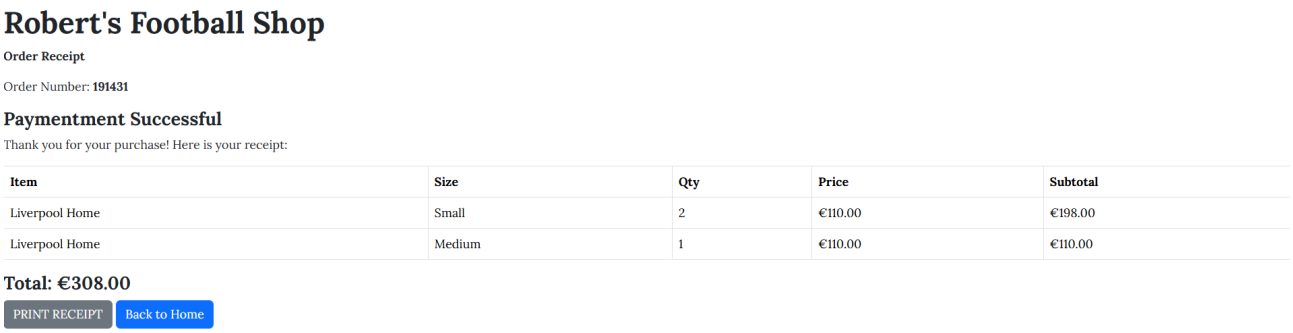
The shopping cart section on the Checkout page displays the items from local memory in a way that is clear and concise to the user.



Once the order has been confirmed then the paymentSuccess.ejs page details a receipt of the items, once again using what was stored in local memory.

After the receipt has been loaded, the local memory is cleared (emptying the cart after confirming order and payment.

The user can also print out a copy of their receipt, which is given a random ‘Order Number’ when the page is rendered.



How this was achieved:

On the shopItems page the addToCart button stores the values from the users selection on the page (such as size, quantity, etc.)

The product types are read from the DOM and are saved in a variable called itemData.

```
// --------------------
// ADD TO CART FUNCTION
// --------------------
function addToCart() {
    //alert("This is from the function!");
    //storing all the information from the DOM in local storage
    //Price and Quantity needs to be Ints (numbers) for Checkout so converted via Number
    let product = document.getElementById("product").innerHTML;         // Product Name
    let type = document.getElementById("type").innerHTML;               // Product Type (jersey, ball, gloves, etc)
    let quantity = Number(document.getElementById("quantity").value);   // Quantity
    let size = document.getElementById("size").value;                   // Size
    let price = Number(document.getElementById("price").innerHTML);     // Price
    let image = document.getElementById("product_image").src;           // Product Image Location
    let ID = document.getElementById("type").dataset.id                 // Product ID from database
    //information above combined into one array
    let itemData = {
        id: ID,
        name: product,
        type: type,
        quan: quantity,
        s: size,
        p: price,
        img: image,

    };
```

The itemData is then saved as a string for the key value pair, using JSON.stringify.
Important to note that the key is saved as product type plus size, meaning a user can add:

Liverpool Home Small
Liverpool Home Xtra Small
Liverpool Away Small
Liverpool Away Xtra Small

etc to the cart, so they are not overwriting previous items in the cart with the same product name (ie Liverpool Home or Liverpool Away).

```
// Save the key with the product name and type (example: Arsenal Away M, Fulham Home XL, etc)
// Allows for multiple of same product with different sizes to be added to cart
let key = type + "" + size;
localStorage.setItem(key, JSON.stringify(itemData));
```

Within checkout.ejs the cart items are ‘unloaded’ and added to the shopping cart display.

The items in local storage are parsed

```
// A loop to go through everything saved in local storage
for (let i = 0; i < localStorage.length; i++) {

    // Gets the storage keys (from local storge)
    let key = localStorage.key(i);
    // Parse the stored JSON objects from the key
    let content = JSON.parse(localStorage.getItem(key));
```

Makes sure price is a number (not a string), quantity is a number (not a string), and that the key value pair is not empty. Any empty key value pair (with quantity of 0) is skipped.

```
// Make sure the objects exist
// Make sure both price and quantity are type int
// Make sure size exists
if (
    !content ||
    typeof content.p !== "number" ||
    typeof content.quan !== "number" ||
    !content.s
) {
    // If all above are true then continue and skip any invalid entries
    continue;
}
```

The discount logic is applied for sizes Small and Xtra Small. This gives a 20% and 10% discount for those sizes, and applies it to the products price.

```
//apply the discount logic based on size
if (discountSize === "Xtra Small") {
    // 20% off for size Xtra Small
    discountRate = 0.20;
} else if (discountSize === "Small") {
    // 10% off for size Small
    discountRate = 0.10;
}

//Calculate the discounted price by taking the discount amount from the original price
let discountedPrice = price * (1 - discountRate);
//Update the total cost for the items (price x quantity)
let total_cost = discountedPrice * content.quan;

// Only show items with a quantity greater than 0
if (content.quan > 0) {
    hasItems = true;
    cost_allItems += total_cost;
```

The below code takes the items with a discount and applies a banner row to the shopping cart table as an alert with the colour ‘warning’ from bootstrap.

Then the cart item row is generated, with the image, name, size, and price inserted into the table. The <del> tags will put a strike-through the original price if there is a discount, and the overall price is shown towards the end of the table row.

```
            //Everything is wrapped in the div "cart-item-container" with data-key so I can remove items from cart
            cartContainer.innerHTML += `
        <div class="cart-item-container" data-key="${key}">
            ${discountRate > 0 ? `
                <div class="alert alert-warning w-100 mb-3">
                    The <strong>${content.type}</strong> got a
                    <strong>${discountRate * 100}% discount</strong>
                    from its original price of
                    <strong>€${price.toFixed(2)}</strong>
                    because it is size <strong>${discountSize}</strong>.
                </div>
            ` : ""}

            <div class="cart-item">
            <div class="col-image"><img src="${content.img}" alt="${content.name}" /></div>
            <div class="col-name"><a href="/shop?rec=${content.id}">${content.type}</a></div>
            <div class="col-size">${content.s}</div>
            <div class="col-price">${discountRate > 0 ? `<del>€${price.toFixed(2)}</del><br>€${discountedPrice.toFixed(2)}`
                : `€${price.toFixed(2)}`
            }
                </div>

            <div class="col-quantity">${content.quan}</div>
            <div class="col-total">€${total_cost.toFixed(2)}</div>

            <div class="col-cancel"><i class="fa-regular fa-circle-xmark"></i></div>
        </div>
    </div>

`;
```

A click event is created for the x icons at the end of the cart. These remove the product from local storage, thus removing them from the shopping cart. The key for that row is removed from local storage and then the page is refreshed to give an updated cart.

```
// Remove item from cart
// Done via click listeners
const cancelIcons = document.querySelectorAll(".col-cancel i");
cancelIcons.forEach(icon => {
    icon.addEventListener("click", (e) => {
        // Gets the key / data of the item in the row which was clicked
        const container = e.target.closest(".cart-item-container");
        const key = container.getAttribute("data-key");
        localStorage.removeItem(key); // Remove the saved Item from Local Storage via its data-key
        location.reload(); // Refresh the page to reload the cart display
    });
});
```

On paymentSuccess.ejs the local storage cart information is used again to formulate the table of items purchased.

The form submit data is used to create the customer details on the receipt / order summary, while the cart information is laid out in a table.

The table is somewhat different here, with no remove item from cart or discount banners shown, but the same loop logic for the key value pairs and discount loop are applied.

```
        // Adds a row to the receipt table for each item and values stored as table data
        receiptContainer.innerHTML += `
    <tr>
        <td>${item.type}</td>
        <td>${item.s}</td>
        <td>${item.quan}</td>
        <td>€${price.toFixed(2)}</td>
        <td>€${total_cost.toFixed(2)}</td>
    </tr>
`;
    }

    // Show the total cost
    totalElement.textContent = "€" + cost_allItems.toFixed(2);
```

After the receipt has been generated, there is also a function to clear the local storage. This way only after a user has submitted to ‘purchasing’ the products does the cart empty.

```
// Clear the cart after receipt loads
localStorage.clear();
```

Item 5 – Object / Array in Javascript

Array use in Carousel:

Within the code structure of the carousel for the home page there is the use of JavaScript array.

An array is used within app.js to store and manage the carousel images.
The server stores all valid image filenames from the /images/jerseys folder.
This array is then randomly shuffled and then the first ten images are selected.

When the array is passed to the front-end (home.ejs) an array-like structure is used within the JavaScript.
A forEach loop is used to access these images via their index.

How this was achieved:

On the server side the let images = [ ] is the array to store all valid image filenames.

```
JS app.js > ...

//-----------------------------
// For Home Page Carousel Images
app.get("/", (req, res) => {

    // Route to the saved images
    const imagesDir = path.join(__dirname, "public/images/jerseys");

    //Creates empty array for the images to be stored before shuffle
    let images = [];

```

The array is shuffled and trimmed.

```
JS app.js > app.get("/") callback
app.get("/", (req, res) => {

        // Shuffle the array randomly
        for (let i = images.length - 1; i > 0; i--) {
            const j = Math.floor(Math.random() * (i + 1));
            [images[i], images[j]] = [images[j], images[i]];
        }

        // Picks the first 10 images after the shuffle
        images = images.slice(0, 10);
```

In home.ejs JavaScript stores all carousel image elements in an array-like structure.

```
<script>

    // Select all images used in the carousel
    const images = document.querySelectorAll(".carousel-img");
```

Indexing is used to control which image is displayed.

```
// Tracks images used in carousel array, starting at index 0
let currentIndex = 0;


// Function to show images at specific index
function showImage(index) {

    // Hides all images by removing the active class
    images.forEach(img => img.classList.remove("active"));

    // Only adds active to the selected images
    images[index].classList.add("active");
}

// For the clicking of the next button
nextBtn.addEventListener("click", () => {

    // Moves to next image and modulo makes index wrap back to 0 at end of array
    currentIndex = (currentIndex + 1) % images.length;

    // Displays the next image
    showImage(currentIndex);
});

// For the previous of the next button
prevBtn.addEventListener("click", () => {

    // Moves to previous image and modulo makes index wrap back to 0 at end of array
    // Adding images.length stops negative numbers
    currentIndex = (currentIndex - 1 + images.length) % images.length;
```

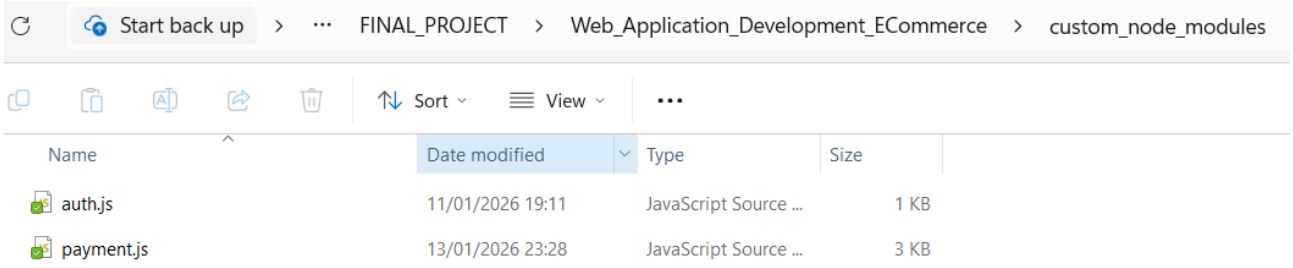
Item 6 – Custom Node Module

Two Custom Node Modules:

I made use of two custom node modules in this project:

auth.js
payment.js

They are saved in the custom_node_modules folder.



The auth.js module is used to authenticate users logging into the website.

The payment.js is used to display the inputs for payment types on the Checkout page.

How this was achieved:

On the server side, in app.js, the custom module auth.js is imported for use as the auth object.

```
JS app.js > ...
//Import authentication custom module
const auth = require("./custom_node_modules/auth.js");
```

It is then used to create users for the website, which are authenticated via the login section.

```
JS app.js > ...
//-------------------------------
//Create Users for the application
auth.createUser("user@123.com", "pass")

//test users work8
console.log(auth.authenticateUser("user@123.com", "pass"));

// End of application users
//-------------------------
```

The payment.ejs custom node module is used to display different payment options.

It is loaded into app.js to access the function getPaymentHTML(), like this:

```
app.js > app.post("/login") callback
//-----------------------------------
//For the payment options in Checkout
//Uses Custom Node Module payment.js
const { getPaymentHTML } = require('./custom_node_modules/payment');

//Get the switch case options from the custom module
app.get('/checkout', (req, res) => {
    res.render('checkout', {
        user: req.session.user || null,
        creditCardFields: getPaymentHTML('credit-card'),
        paypalFields: getPaymentHTML('paypal'),
        stripeFields: getPaymentHTML('stripe'),
        cryptoFields: getPaymentHTML('crypto')
    });
});

// End of Payment Options
//-----------------------
```

Within checkout the user is given a dropdown menu when entering payment. The options are as above : credit card, paypal, stripe, and cryptocurrency.

The input box on the checkout page changes depending on which option is selected. Therefore all payment-related markup and logic are kept separate from app.js and checkout.ejs.

Within payment.js a switch statement is used to define all the cases. Both credit card and stripe have the same input layout, but paypal and cryptocurrency are different.

These then fit into the form for the checkout, with the code shown on the next page.

payment.ejs file with switch statements

custom_node_modules > JS payment.js > getPaymentHTML

```
function getPaymentHTML(method) {
    switch (method) {
        case "credit-card":
        case "stripe":
            return `
                <div id="credit-card-stripe-fields" class="payment-fields">
                    <div class="row g-3 mb-3">
                        <div class="col-md-6">
                            <input class="form-control" placeholder="Name on Card">
                        </div>
                        <div class="col-md-3">
                            <input class="form-control" placeholder="MM/YY">
                        </div>
                        <div class="col-md-3">
                            <input class="form-control" placeholder="CVV">
                        </div>
                        <div class="col-12">
                            <input class="form-control" placeholder="Card Number">
                        </div>
                    </div>
                </div>`;
        case "paypal":
            return `
                <div id="paypal-fields" class="payment-fields">
                    <div class="row g-3">
                        <div class="col-12">
                            <input class="form-control" placeholder="PayPal Email:">
                        </div>
                    </div>
                </div>`;
        case "crypto":
            return `
                <div id="crypto-fields" class="payment-fields">
                    <div class="row g-3 mb-3 align-items-center">
                        <div class="col-md-2">
                            <label>Network:</label>
                        </div>
```

Item 7 – Post and Get Requests

Get Requests:

GET requests are used throughout for the rendering of pages. All accessible pages are saved in .ejs format and therefore GET requests are required.

Pages using the GET requests are:

- about.ejs
- contactUs.ejs
- jerseyShop.ejs
- miscShop.ejs
- shopEntire.ejs
- checkout.ejs
- home.ejs
- shopItems.ejs
- login.ejs

The search bar function also works using the GET requests. There are two within the app.js.

The first is to search via item ID and this is used for rendering items on the shopItems.ejs page.
It makes use of WHERE to find by ID.

The second is to search via product or club name, allowing users to input “arsenal” into the searchbar and be directed to the shopItems.ejs page for that product.
It makes use of the LIKE to find by name.

Post Requests:

Pages using POST requests are:

- PaymentSuccess.ejs

This is done after the form validation on the checkout.ejs page to ensure sensitive user information can not be resubmitted. The information is saved in the server for the session and can be used elsewhere, in this case from the checkout to the receipt.

How this was achieved:

Within app.js the routes to pages are saved as either GET METHOD or POST METHOD.

GET requests retrieve data from the server. POST requests send data to the server to create or update resources.

A common GET request to render a page is like this:

```
app.js > app.get("/jerseyShop") callback

//Route to the Jersey Shop Page (GET Method)
app.get("/jerseyShop", (req, res) => {
    res.render("jerseyShop");
});
// End of Route for Jersey Shop Page (GET Method)


//Route to the Misc Items Shop Page (GET Method)
app.get("/miscShop", (req, res) => {
    res.render("miscShop");
});
// End of Route for Misc Items Shop Page (GET Method)
```

This allows for linking the jerseyShop.ejs page by including /jerseyShop within a link, or the miscShop.ejs by including /miscShop within a link.

The search by ID using WHERE GET request is as follows:

```
app.js > app.get("/shop") callback > db.query("SELECT * FROM product_data.database WHERE ID = ?") callback

//-----------------------------------------------------------------------
//GET METHOD /shop USED TO CREATE SHOP LAYOUT PAGES USING ID FROM DATABASE
app.get("/shop", function (req, res) {
    const ID = req.query.rec;
    db.query("SELECT * FROM product_data.database WHERE ID = ?", [ID], function (err, rows, fields) {
        if (err) {
            console.error("Error getting data from database.", err);
            res.status(500).send("Error retreiving data from database.");
        } else if (rows.length === 0) {
            console.error("No rows found for ID $[ID]");
            res.status(404).send("Product not found");
        } else {
            //To monitor the application while developing it
            console.log("Data retreived from the database.");
            console.log(rows[0].Club);
            console.log(rows[0].Version);
            console.log(rows[0].Price);
            console.log(rows[0].League);
            console.log(rows[0].ID);

            //Variables from Database
            const clubName = rows[0].Club;
            const versionName = rows[0].Version;
            const price = rows[0].Price;
            const league = rows[0].League;
            const manufact = rows[0].Manufacturer;
            const images = rows[0].Image;
            const crest = rows[0].Crest;
            const ID = rows[0].ID;
```

The database query selects items from the database based on their ID number.

It then stores the variables for each item index into a number of different const variables associated with the value from the row (price, image, league, etc)

It then renders the shopItems.ejs based on the values stored in those variables, so for whatever ID is loaded on the page the products and images associated with that ID will be shown.

```
app.js > app.get("/shop") callback > db.query("SELECT * FROM product_data.database WHERE ID = ?") callback
app.get("/shop", function (req, res) {
    db.query("SELECT * FROM product_data.database WHERE ID = ?", [ID], function (err, rows, fields) {
                function (err2, altRows) {

            //Inject data into a HTML page - Render the shopItems page with the information stored
            res.render("shopItems.ejs", {
                myClub: clubName, myVersion: versionName, myPrice: price, myLeague: league,
                myManufacturer: manufact, myImage: images, myCrest: crest, altLink: altLink,
                altVersion: altVersion, myID: ID,
            });
                }
            );
        }
    });
});

// End of SHOP LAYOUT GET Method
//-----------------------------
```

The GET request using LIKE is as follows:

```
app.js > app.get("/search") callback > db.query() callback

//-----------------------------------------------------------------------------------------------------------
//GET METHOD /search FOR SEARCHING BY PRODUCT NAME OR CLUB NAME / ITEM TYPE AS PER Club and League FROM DATABASE
app.get("/search", function (req, res) {
    const searchTerm = req.query.q;

    const sql = `
        SELECT ID
        FROM product_data.database
        WHERE Club LIKE ? OR League LIKE ?
        LIMIT 1
    `;

    db.query(
        sql,
        [`%${searchTerm}%`, `%${searchTerm}%`],
        function (err, rows) {
            if (err) {
                console.error("Search error:", err);
                return res.redirect("/?searchError=1"); // redirect to homepage with error flag
            }

            if (rows.length === 0) {
                return res.redirect("/?searchError=1"); // redirect if no products found
            }

            res.redirect(`/shop?rec=${rows[0].ID}`);
        }
    );
});

// End of GET METHOD /search
//-------------------------
```

Instead of using WHERE to get the exact ID number, this uses LIKE which means even typing part of the name for the club or league (live for Liverpool, bar for Barcelona, chel for Chelsea, etc) will return the product shopItems.ejs page for that search.

Error handling is included to redirect back to the home page if no product exists in the database or if an item is input into the searchbar which does not exist.

The POST request for paymentSuccess.ejs is as follows:

```
app.js > app.post("/paymentSuccess") callback

//Route to Payment / Order Successful after Checkout Page (POST METHOD)
app.post("/paymentSuccess", (req, res) => {
    // Generate a random number
    const orderNumber = Math.floor(Math.random() * 900000 + 100000); // 6-digit number

    // Capture all form data from checkout.ejs
    const formData = {
        firstName: req.body.firstName,
        lastName: req.body.lastName,
        addressLine1: req.body.addressLine1,
        addressLine2: req.body.addressLine2,
        city: req.body.city,
        zip: req.body.zip,
        email: req.body.email,
        paymentMethod: req.body.payment
    };

    //Generates the order number when page is rendered
    res.render("paymentSuccess", {
        orderNumber,    // pass as object to avoid error
        formData        // add the user data input from checkout.ejs
    });
});
// End of Route for Payment / Order Successful after Checkout Page (POST METHOD)
```

It makes use of app.post instead of app.get to specify that it is a POST request.

Within there is also a random number generator for creating an ‘Order Number.’

The const formData is where all the user inputs are stored from checkout.ejs before being used again on paymentSuccess.ejs.

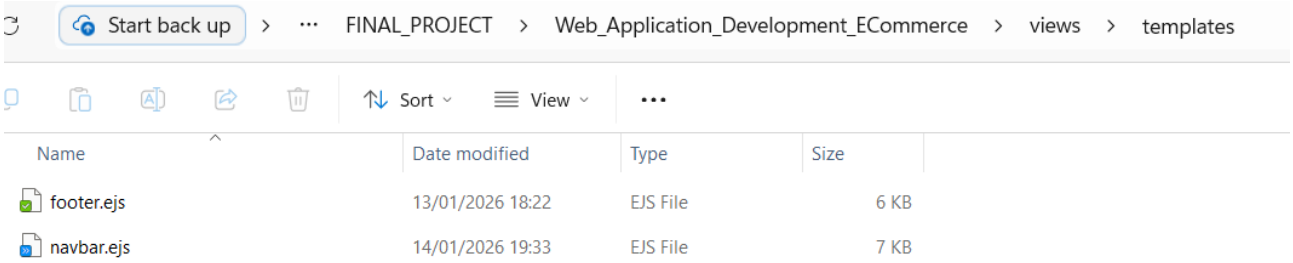
```
paymentSuccess.ejs > html > body > div#receipt > h3
<html>
<body>
    <div id="receipt">
        <!-- Customer Info -->
        <!-- Via POST Method on checkout.ejs form-->
        <h3>Customer Details</h3>
        <p>
            Name: <strong><%= formData.firstName %> <%= formData.lastName %></strong><br>
            Address: <strong><%= formData.addressLine1 %>
                <% if (formData.addressLine2) { %> <%= formData.addressLine2 %> <% } %>,
                <%= formData.city %>, <%= formData.zip %></strong><br>
            Email: <strong><%= formData.email %></strong><br>
            Payment Method: <strong><%= formData.paymentMethod %></strong>
        </p>
        <!-- End of Customer Info -->
```

Item 8 – Static and Dynamic Content

Both static and dynamic content are used throughout the project.

Static Content:

Main examples of the static content are the navbar and the footer. Both files are saved in /views/templates and are loaded into the pages.

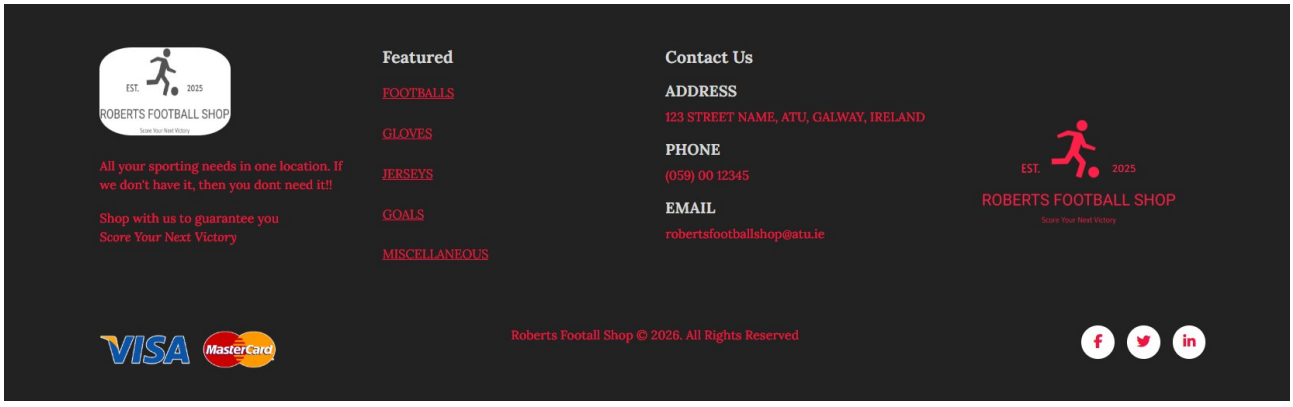


They appear as follows on the pages:

navbar.ejs

The majority of the navbar is static, with the logo, shop name, links, search function, and cart icon. The only dynamic part is the login icon, which changes when the user is logged in.

footer.ejs



All of the footer is static, with the logos, text, links, copyright info, payment images, and social media icons all appearing the same on every page. While the social media icons do change colour with a hover effect, they are still considered static content as the HTML content itself does not change – they are the same for every visitor of the site.

Dynamic Content:

As mentioned, the login icon in the navbar is dynamic as it depends on the user's session data (whether they are logged in or not).

Another example of dynamic content is the shopItems page. The product information like name, price, club, and image are loaded from the database. Each product page displays different data depending on the ID queried from the database.

In the checkout page, the shopping cart total and product details are dynamically inserted via JavaScript from the user's cart (local storage).

How this was achieved:

The static content from navbar.ejs and footer.ejs are included at the top of the body in each page they appear on.

```
<body>
    <!-------------------->
    <!-- Navbar Section -->
    <%- include('templates/navbar') %>

    <!-- End of Navbar Section -->
    <!--------------------------->

```

```
        <!------------------->
        <!-- Footer Section-->
        <%- include('templates/footer') %>

        <!-- End of Footer Section-->
        <!-------------------------->
```

The <%- include ('path/to/file') %> is an ejs command which instructs the server to insert the content from another file into the current template. Therefore by having navbar.ejs and footer.ejs as distinct files they can be inserted on multiple pages with the same look.

To update the files later they just need to be updated once, and it saves having repetitive code on multiple pages.

The content within the files are static as they do not change.

Below is an extract from the footer.ejs file which makes use of standard HTML code and links to images without the use of any dynamic features. Any effects which change the visual outlook, such as when hovering over the social media icons, happens via CSS.

```
views > templates > footer.ejs > footer > div.row.container.mx-auto.pt-3.align-items-stretch
<!-- Footer Section-->

<!--Main footer container-->
<footer>

    <!-- ...
    <div class="row container mx-auto pt-3 align-items-stretch">

        <!-- Column 1: Brand logo (fixed size) and short description -->
        <!-- Responsive sizing: Large = 3 cols, Medium = 6, Small = full width -->
        <div class="footer-one col-lg-3 col-md-6 col-12">
            <img id="footer_small_logo" src="/public/images/logo/logo-grayscale-transparent.png" alt="logo"
                width="100px">
            <!--Padding top adds spacing between logo and text via bootstrap -->
            <p class="pt-4">All your sporting needs in one location. If we don't have it, then you dont need it!!
            </p>
            <p> Shop with us to guarantee you<br>
                <em>Score Your Next Victory</em>
            </p>
        </div>
        <!-- End of Column 1: Brand logo -->
```

The dynamic content is different. Using shopItems.ejs page as the example:

the page title is dynamic as it changes depending on which product is loaded
It gets the club and version from app.js based on which ID is selected from the database.

```
views > shopItems.ejs > html
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width" />

    <!-- makes page title from db depending on what club and version is shown -->
    <title>
        <%= myClub %>
            <%= myVersion %>
    </title>
```

For the product image, dynamic content is again used as <%= imageFolder %>/<%= myImage %> uses the database info from app.js to determine which folder and image to use.
The correct product image is automatically selected.

```
<!--This will select the image based on the imageFolder from the loop above (jerseys or misc_items)-->
<div class="product-image-wrapper">
    <img id="product_image" src="/public/images/<%= imageFolder %>/<%= myImage %>"
        class="img-fluid mt-4 product-image" alt="Product Image">
</div>
<!-- End of Item Image -->
```

The product info for club, version, league, manufacturer, and price are all taken from the database (via app.js).

```
<!-- Product info -->
<!-- id used for Javascript addToCart Button -->
<h2 id="type" data-id="<%= myID %>">
    <%= myClub %>
        <%= myVersion %>
</h2>

<!-- League Label (Displays League or Product depending on item - Jersey or Misc)-->
<!-- The leagueLabel was defined in the for loop with const miscProducts -->
<p><strong>
        <%= leagueLabel %>
    </strong>
    <%= myLeague %>
</p>
<p><strong>Manufacturer: </strong>
    <%= myManufacturer %>
</p>
<!-- End of Product info -->


<!-- Product Price -->
<!-- text-danger = applies a red colour -->
<h3 class="text-danger">
    <!-- id used for Javascript addToCart Button -->
    €<span id="price">
        <%= myPrice %>
    </span>
</h3>
<!-- End of Product Price -->
```

The alternative jersey option, which is loaded in from app.js, which will display a like to the alternative jersey's only. So if you are viewing Liverpool home, it will link to Liverpool away, etc.

```
<!-------------------------------------------------------------------------->
<!-- Alternative Products Link (exampe Arsenal away if on Arsenal home page) -->
<!-- if an alternative link exists (done in app.js) -->
<% if (altLink) { %>
```

(altLink) from app.js
Gathers the information using a WHERE query to find the alternative version and store the information for the link on the shopItems.ejs page.

```
//QUERY: will look in the database for the alternate version (Home ↔ Away)
db.query(
    "SELECT ID, Version FROM product_data.database WHERE Club = ? AND Version != ? LIMIT 1",
    [clubName, versionName],
    function (err2, altRows) {

        if (err2) {
            console.error("Error fetching alternate version", err2);
            res.status(500).send("Error retreiving alternate product.");
            return;
        }

        // Build link if alternate exists (back to shopItems via the ID)
        const altLink = altRows.length > 0
            ? `/shop?rec=${altRows[0].ID}`
            : null;

        const altVersion = altRows.length > 0
            ? altRows[0].Version
            : null;
```

Dynamic sizing options based on database information. So if the League column in the database matching the product ID says gloves, there will be different sizes compared to balls.

```
<select id="size" name="size">
    <!-- ...
    <%# Size options for Footballs using the else if %>
        <% if (myLeague === "Ball" ) { %>
            <option value="Size 3">Size 3</option>
            <option value="Size 4">Size 4</option>
            <option value="Size 5" selected>Size 5</option>

            <%# Size options for Goalkeeper Gloves using the else if %>
                <% } else if (myLeague === "Gloves" ) { %>
                    <option value="Xtra Small">Junior</option>
                    <option value="Small">Adult Small</option>
                    <option value="Medium" selected>Adult Medium</option>
                    <option value="Large">Adult Large</option>

                    <%# Size Options for Shin Pads using the else if %>
                        <% } else if (myLeague === "Shin Pads" ) { %>
                            <option value="Small">Small</option>
                            <option value="Medium">Medium</option>
                            <option value="Large" selected>Large</option>

                            <%# Default Value for any uncategorised Items %>
                                <% } else if (myLeague === "Goals" ) { %>
                                    <option value="Standard">Standard</option>

                                    <%# Size Options for Jerseys (different league names -
                                        Premier League, La Liga, etc) using if loop %>
                                        <% } else { %>
                                            <option value="Xtra Small">XS</option>
                                            <option value="Small">S</option>
                                            <option value="Medium" selected>M</option>
                                            <option value="Large">L</option>
```

Within the addToCart function (JavaScript part of shopItems.ejs file) the variables are dynamic as they change depending on which product is on display. When the addToCart button is clicked the current dynamic data is collected for saving in local storage.

This is done with getElementByID and saving the information into a variable, which is then saved in the key value pair in local storage.

```
// --------------------
// ADD TO CART FUNCTION
// --------------------
function addToCart() {
    //alert("This is from the function!");
    //storing all the information from the DOM in local storage
    //Price and Quantity needs to be Ints (numbers) for Checkout so converted via Number
    let product = document.getElementById("product").innerHTML;         // Product Name
    let type = document.getElementById("type").innerHTML;               // Product Type (jersey, ball, gloves, etc)
    let quantity = Number(document.getElementById("quantity").value);   // Quantity
    let size = document.getElementById("size").value;                   // Size
    let price = Number(document.getElementById("price").innerHTML);     // Price
    let image = document.getElementById("product_image").src;           // Product Image Location
    let ID = document.getElementById("type").dataset.id                 // Product ID from database
    //information above combined into one array
    let itemData = {
        id: ID,
        name: product,
        type: type,
        quan: quantity,
        s: size,
        p: price,
        img: image,

    };
```

Item 9 – Templates in Node

Node Templates:

Some of the templates used in this project were:

navbar.ejs
footer.ejs

They are inserted on multiple pages using **include( )** for ejs

How this was achieved:

In app.js the below snippet of code instructs Node to use EJS as the template engine, while using the file path ‘views’ to locate files.

Files stored elsewhere have been defined with the filepath ‘public’

```
JS app.js > app.post("/login") callback
35
36  // View engine - uses __dirname for absolute path in directory
37  app.set("view engine", "ejs");
38  app.set("views", path.join(__dirname, "views"));
39
40  // Static files - uses __dirname for absolute path in directory (CSS, JS, images, etc.)
41  app.use("/public", express.static(path.join(__dirname, "public")));
42
```

navbar.ejs and footer.ejs using the include ( ) function from ejs.

```
21  <body>
22      <!-------------------->
23      <!-- Navbar Section -->
24      <%- include('templates/navbar') %>
25
26      <!-- End of Navbar Section -->
27      <!--------------------------->
28
```

```
225
226         <!------------------->
227         <!-- Footer Section-->
228         <%- include('templates/footer') %>
229
230         <!-- End of Footer Section-->
231         <!-------------------------->
232
```

More displays of templating using Node is when Node replaces <%= XXXX %> placeholders with actual values, often gotten from the database although sometimes also defined in custom modules or defined within app.js.

An example is the product data within app.js, which get inserted in app.js.

```
//Inject data into a HTML page - Render the shopItems page with the information stored
res.render("shopItems.ejs", {
    myClub: clubName, myVersion: versionName, myPrice: price, myLeague: league,
    myManufacturer: manufact, myImage: images, myCrest: crest, altLink: altLink,
    altVersion: altVersion, myID: ID,
});
    }
);
```

In shopItems the myClub and myVersion were defined from the database within app.js.

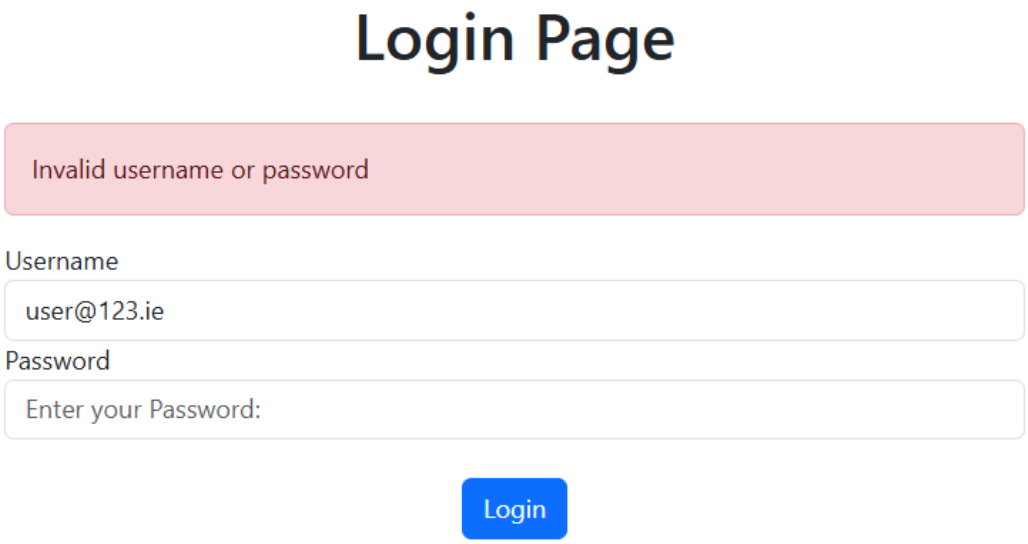
```
<!-- Product info -->
<!-- id used for Javascript addToCart Button -->
<h2 id="type" data-id="<%= myID %>">
    <%= myClub %>
        <%= myVersion %>
</h2>
```

Item 10 – Error Message

Error handling:

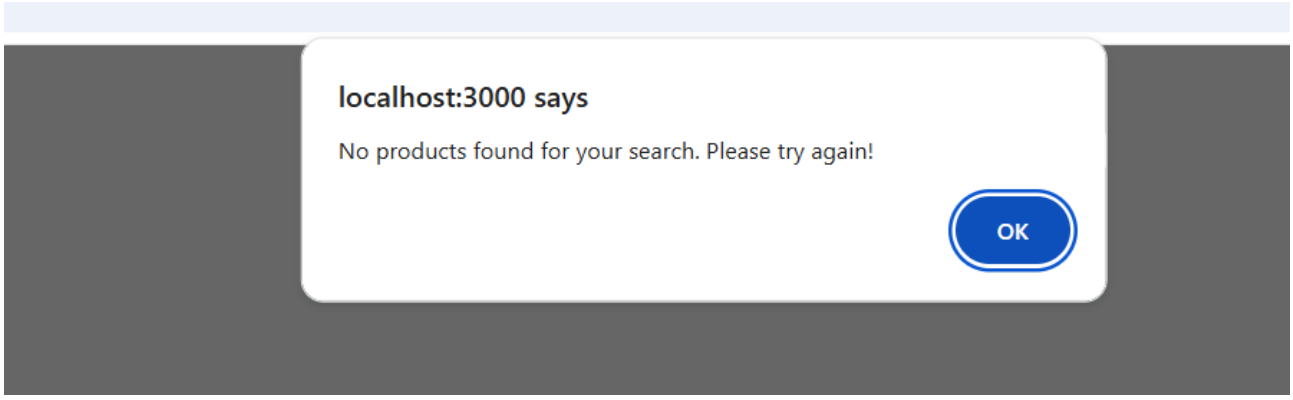
On the login page an error message appears if the user inputs the wrong username or password.

When the page reloads, the previously input username is retained



The search function in the navbar also contains error handling.

If the user inputs a search term which is not available within the database, then a popup alert will appear on the page stating “No products found.” When the user clicks OK they are redirected back to the home page.



How this was achieved:

Within the login.ejs page the server will pass an error if the wrong credentials have been entered. This links back to the auth.ejs custom node module authenticateUser function.

The alert appears using the danger colour from bootstrap to clearly indicate to the user that the wrong details have been input.

```
<!--Error Message if Node.js passes an error-->
<% if (error) { %>
    <div class="alert alert-danger">
        <!-- Outputs the error message from the server -->
        <%= error %>
    </div>
    <% } %>
```

However, to ensure ease of use for the user the username is retained even after an error has appeared.

This is achieved via the if else statement in app.js.

The else part of if (authenticated) will render the login page again and display the error text “Invalid username or password”

The username: username part is what retains the previously input username.

In the initial app.get(“/login, …) the display has the error as null so it does not appear on first time page loading, and the username: “ “ leaves the username input blank.

app.js > app.post("/login") callback

```javascript
//-------------------------------
//Route to Login Page (GET METHOD)
app.get("/login", (req, res) => {
    res.render("login", { error: null, username: "" });
});

//Route to handle Login Form Submission - (POST METHOD)
app.post("/login", function (req, res) {
    const username = req.body.username;
    const password = req.body.password;

    //authenticate the username and password
    const authenticated = auth.authenticateUser(username, password);
    console.log(authenticated);

    if (authenticated) {
        // store user in session
        req.session.user = username;

        //Logs to the console that the login was a success
        //If login is successful then directed to the home page (/)
        console.log("Successful Authentication");
        res.redirect("/");
    }
    else {
        //If login fails the login page renders again
        //Logs to the console that the login failed
        //Redirects to the login page with the previously input username still shown
        //Gives an alert message saying invalid username or passowrd
        console.log("Failed Authentication");
        res.render("login", {error: "Invalid username or password", username: username });
    }
});

// End of Login Route Methods
//--------------------------
```

For the navbar search function, a GET request is used. Within the request an if statement handles the potential of errors.

If the user inputs an invalid search terms they are redirected back to the home page.

if (err) {

If the query runs correctly but returns no matching products then the user is redirected to the home page.

if (rows.length === 0) {

app.js > ...

```javascript
//-----------------------------------------------------------------------------------------------------------
//GET METHOD /search FOR SEARCHING BY PRODUCT NAME OR CLUB NAME / ITEM TYPE AS PER Club and League FROM DATABASE
app.get("/search", function (req, res) {
    const searchTerm = req.query.q;

    const sql = `
        SELECT ID
        FROM product_data.database
        WHERE Club LIKE ? OR League LIKE ?
        LIMIT 1
    `;

    db.query(
        sql,
        [`%${searchTerm}%`, `%${searchTerm}%`],
        function (err, rows) {
            if (err) {
                console.error("Search error:", err);
                return res.redirect("/?searchError=1"); // redirect to homepage with error flag
            }

            if (rows.length === 0) {
                return res.redirect("/?searchError=1"); // redirect if no products found
            }

            res.redirect(`/shop?rec=${rows[0].ID}`);
        }
    );
});

// End of GET METHOD /search
//-------------------------
```

Item 11 – MySQL Database

MySql:

The MySQL database was downloaded and used in this project for all the product information.

Database Name: g00474404
Password: root (all lowercase)
User: root
Host: localhost

The columns within the database included:

- ID – number for product to use as primary key
- Club – name of the product, for example Arsenal, Liverpool, or Total 90 for a football.
- Version – this allowed for both Home and Away jerseys
- League – this defined the league the jersey is from (Premier League, La Liga, International) or product type if it was a Ball, Gloves, Goal, Shin Pads.
- Manufacturer – this is for the manufacturer of the products
- Crest – name of the image file saved for the crest / logo of the product
- Image – main product image

How this was achieved:

Within the database folder the db.js file handles connecting to the MySQL database.

database > JS db.js > ...

```javascript
//Connect to Database
const mysql = require('mysql');

//Create a connection to MySQL database
const connection = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: 'root',
    database: 'g00474404',
});

//Connect to database
connection.connect((err) => {
    if (err) {
        console.error('Error connecting to database', err);
    } else {
        console.log('Connected to database!');
    }
});

//Export for use elsewhere
module.exports = connection;
```

Within app.js the route to the database is connected, allowing the server to access the stored information from db.js.

```
JS app.js > ...
//Connect to the db.js file
const db = require("./database/db");
```

Item 12 – Bootstrap Version 5

Bootstrap:

Bootstrap version 5 was included in each page for the project.

How this was achieved:

In the head section Bootstrap for CSS was included:

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width" />

    <title>Login Page</title>

    <!--Importing Bootstrap 5 for CSS-->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">

    <!--Linking CSS file-->
    <link rel="stylesheet" href="./css/style.css">

    <!--Font Awesome-->
    <script src="https://kit.fontawesome.com/8875caf3cf.js" crossorigin="anonymous"></script>

</head>
```

Towards the end of the <body> section Bootstrap for JavaScript was included:

```
    <!--Importing Bootstrap 5 for JS-->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>

</body>

</html>
```

Additional

Folder Structure Layout

```
Web_Application_Development_ECommerce/
│
├── app.js
├── package.json
├── package-lock.json
├── codeswing.json
├── .gitignore
│
├── custom_node_modules/
│   ├── auth.js
│   └── payment.js
│
├── database/
│   ├── Database.csv
│   ├── Database with References.csv
│   └── db.js
│
├── document/
│
├── node_modules/
│
├── public/
│   ├── css/
│   │   └── style.css
│   ├── images/
│   │   ├── background/
│   │   ├── banner/
│   │   ├── brands/
│   │   ├── crest/
│   │   ├── favicon/
│   │   ├── featured_sections/
│   │   ├── jerseys/
│   │   ├── logo/
│   │   ├── misc_items/
│   │   └── payment/
│   └── script.js
│
└── views/
    ├── templates/
    │   ├── navbar.ejs
    │   └── footer.ejs
    ├── about.ejs
    ├── checkout.ejs
    ├── contactUs.ejs
    ├── home.ejs
    ├── jerseyShop.ejs
    ├── login.ejs
    ├── miscShop.ejs
    ├── paymentSuccess.ejs
    ├── shopEntire.ejs
    └── shopItems.ejs
```

Image References

The references for all the images used are saved within the project folder -

/database/DatabaseWithReferences.csv

There are 2 additional column in this excel sheet which contain the url for where I downloaded the images.